



Building AI Agents with MCP: A Practical Guide to Multi-Agent Systems

19 min read · Oct 1, 2025



Furkhan Suhail

Follow

Listen

Share

More

When people hear *AI agents*, they often think of highly complex systems for stock trading, robotics, or research copilots. But the core concepts can be hard to digest if we jump straight into those domains.

The landscape of AI development is shifting from monolithic models to modular, intelligent systems. AI Agents represent this evolution — autonomous programs that can perceive their environment, make decisions, and take actions to achieve specific goals.

That's why, in this article, we'll explore what AI Agents are, introduce the Model Context Protocol (MCP), and we'll use the **simplest possible case: a calculator (A practical multi-agent calculator system)**. By sticking to basic math operations — addition, subtraction, multiplication, and division — we remove the distraction of domain-specific complexity. This lets us focus purely on how **AI agents** and **MCP (Model Context Protocol)** servers work together to process requests, route decisions, and generate explanations. and that demonstrates these concepts in action.

Think of this project as a **teaching sandbox**: simple enough to follow, yet powerful enough to demonstrate the architecture behind real-world multi-agent AI systems. This is for someone who wants to learn about AI Agents and MCP and have no knowledge on what they are and how they work.

Prerequisite: python and API knowledge

What Are AI Agents?

An AI Agent is a software program that uses artificial intelligence to autonomously perform tasks on behalf of users. Unlike traditional applications that follow rigid programmed instructions, AI Agents can:

- **Perceive** their environment through various inputs (user queries, APIs, databases)
- **Reason** about the best course of action using language models
- **Act** by calling tools, APIs, or other services
- **Learn** from outcomes to improve future performance

Why Use AI Agents?

AI Agents solve several key challenges in modern software development:

1. **Task Automation:** Agents can handle complex, multi-step workflows without constant human intervention.
2. **Dynamic Decision Making:** Rather than hardcoding every possible scenario, agents adapt their behavior based on context.
3. **Tool Integration:** Agents can orchestrate multiple specialized tools, each doing one thing well.
4. **Natural Language Interface:** Users interact with agents conversationally, making complex systems accessible.
5. **Scalability:** By breaking systems into specialized agents, you can scale and maintain components independently.

Real-World Agent Capabilities

Modern AI Agents can:

- Analyze data and generate insights
- Make API calls to external services
- Query databases and process information

- Generate and execute code
- Coordinate with other agents
- Maintain conversation context
- Handle error recovery and retries

While the idea of **agentic AI** is exciting, the real challenge is **coordination**. An agent isn't just one big model doing everything — it's a system that decides *which specialized tool* should solve a given problem. This is where the **Model Context Protocol (MCP)** comes in. MCP acts as the backbone for agentic systems, defining how agents talk to different tools and services in a standardized way. By following MCP, agents become **modular, extensible, and reliable**: you can plug in a new tool (like a calculator, a stock API, or a weather service) without rewriting the entire agent. In short, **MCP is the glue that makes agentic AI practical and scalable**.

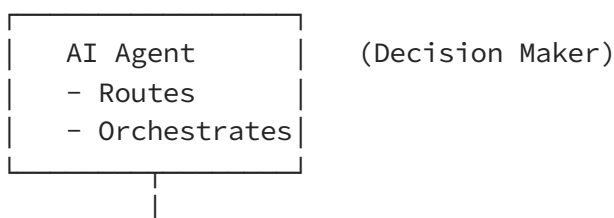
Understanding the Model Context Protocol (MCP)

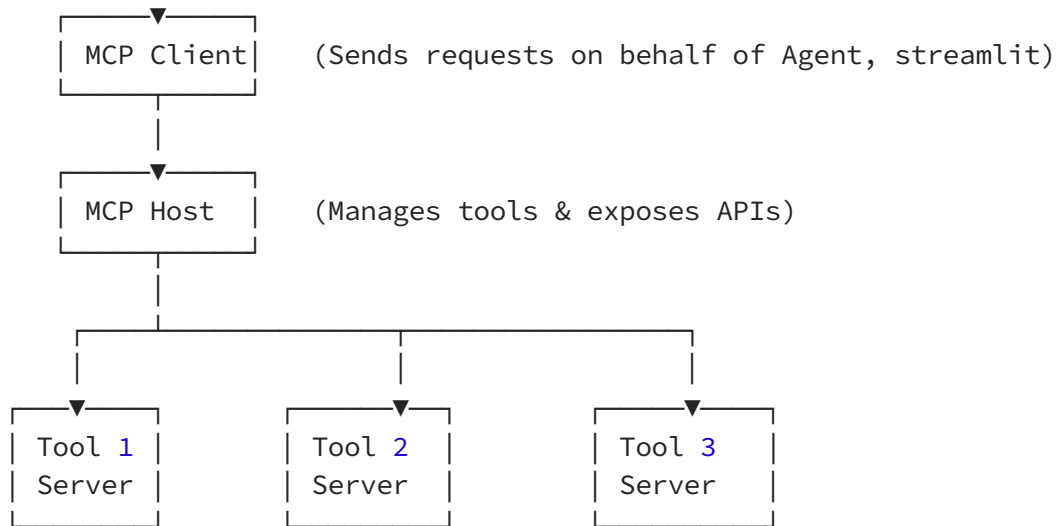
The Model Context Protocol is a standardized way for AI applications to interact with external tools and data sources. Think of MCP as a universal adapter that lets AI models connect to various services in a consistent way.

Why MCP Matters

Before MCP, every tool integration was custom-built. If you wanted your AI to check weather, send emails, or query a database, you'd write unique code for each integration. MCP solves this by providing:

- **Standardized Interface:** Tools expose themselves through a common protocol
- **Discoverability:** Agents can learn what tools are available and how to use them
- **Modularity:** Add or remove tools without changing agent code
- **Separation of Concerns:** Tool servers handle specific tasks while agents orchestrate





Project Overview: Multi-Agent Calculator System

We'll build a calculator system that demonstrates key AI Agent concepts:

- **Two specialized MCP servers** providing math operations
- **Two AI agents** (Kimi K2(Hugging Face) and OpenAI GPT) handling different operations
- **An orchestration layer** routing requests to the right agent
- **Natural language explanations** of mathematical results

System Architecture

```
User → Streamlit UI → Agent API → [Agent Router]
                                ↳ Kimi K2 Agent → MCP Server 1 (Add/Sub)
                                ↳ OpenAI Agent → MCP Server 2 (Multipl
```

Building the MCP Servers

MCP servers are specialized services that perform specific tasks. Each server exposes tools through HTTP endpoints.

MCP Server 1: Basic Operations

```

# mcp_server_1.py
# =====
# MCP SERVER 1: Basic Operations (Addition & Subtraction)
# =====

# This file represents one MCP (Model Context Protocol) server.
# In an agentic system, the "MCP server" is responsible for exposing
# a set of specialized tools (in this case, math operations)
# through well-defined APIs.
#
# The AI Agent does NOT do the math itself. Instead, it routes
# user requests to MCP servers, which actually perform the operation
# and return structured results.

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

# Create a FastAPI app instance for MCP Server 1
app_mcp1 = FastAPI(title="MCP Server 1 - Basic Operations")

# -----
# REQUEST & RESPONSE MODELS (Pydantic)
# -----
# Pydantic BaseModel is used to define the structure of the data
# this server expects (input) and returns (output).
# FastAPI automatically validates incoming requests against these models.

class OperationRequest(BaseModel):
    # Input data schema for a math operation
    # The request must contain two numbers.
    number1: float
    number2: float

class OperationResponse(BaseModel):
    # Output data schema for a math operation
    # The response will always include:
    # - result: numeric result of the operation
    # - operation: the name of the operation (addition/subtraction)
    # - explanation: a human-readable string of what was done
    result: float
    operation: str
    explanation: str

# -----
# TOOL 1: ADDITION
# -----
# In MCP terminology, a "tool" is just an endpoint that performs a
# specific function. Here, the "add" tool adds two numbers together.
#
# The AI Agent can invoke this tool by sending a POST request to
# /tools/add with a JSON body matching the OperationRequest schema.

```

```

@app_mcp1.post("/tools/add", response_model=OperationResponse)
async def add_numbers(request: OperationRequest):
    """Addition tool - adds two numbers"""
    # Validation: FastAPI + Pydantic ensure that request.number1
    # and request.number2 are valid floats before reaching this line.
    result = request.number1 + request.number2

    # The tool returns a structured response matching OperationResponse.
    return {
        "result": result,
        "operation": "addition",
        "explanation": f"Added {request.number1} and {request.number2}"
    }

# -----
# TOOL 2: SUBTRACTION
# -----
# Another tool exposed by this MCP server. It subtracts the second
# number from the first.
#
# Again, the agent will call this endpoint via POST /tools/subtract
# with the appropriate JSON input.

@app_mcp1.post("/tools/subtract", response_model=OperationResponse)
async def subtract_numbers(request: OperationRequest):
    """Subtraction tool - subtracts second number from first"""
    result = request.number1 - request.number2
    return {
        "result": result,
        "operation": "subtraction",
        "explanation": f"Subtracted {request.number2} from {request.number1}"
    }

# -----
# HOW TOOLS ARE INVOKED
# -----
# 1. The AI Agent receives a user query (e.g., "subtract 7 from 10").
# 2. The Agent decides which MCP server/tool to use.
# 3. It makes a POST request to the correct endpoint, for example:
#     POST http://localhost:8001/tools/subtract
#     JSON body: {"number1": 10, "number2": 7}
# 4. MCP Server 1 validates the request, performs the operation,
#     and returns a JSON response with the result and explanation.
# 5. The Agent then adds an LLM-generated explanation if needed
#     before returning the final answer to the user.

```

Key Concepts:

- **Tool Encapsulation:** Each endpoint is a discrete tool performing one function
- **Standardized I/O:** Consistent request/response models make tools predictable
- **Stateless Design:** Each request is independent, enabling horizontal scaling

SideQuest: What is Pydantic?

Pydantic is a Python library for **data validation and settings management**. It uses **Python type hints** to make sure that the data you pass around in your application is:

- **Valid** (e.g., a float really is a float, not a string).
- **Well-structured** (follows a defined schema).
- **Easy to serialize/deserialize** (convert between JSON ↔ Python objects).

It's very popular in **FastAPI**, because FastAPI automatically integrates with Pydantic to:

- Validate incoming API requests.
- Generate API documentation (Swagger/OpenAPI).
- Prevent runtime errors due to unexpected input.

Why are we using Pydantic in this project?

In your **agent + MCP project**, Pydantic is used to define **request** and **response** models for APIs.

```
class OperationRequest(BaseModel):  
    number1: float  
    number2: float  
  
class OperationResponse(BaseModel):  
    result: float  
    operation: str  
    explanation: str
```

Here's why:

1. Validation

If the client sends `"number1": "abc"`, Pydantic will throw an error instead of breaking your server. prevents invalid data from reaching your business logic.

2. Consistency

All MCP servers return results in the same format (result, operation, explanation). The Agent API can trust that responses will follow this schema.

3. Serialization

Requests and responses are automatically converted between **Python objects** ↔ **JSON**. This reduces boilerplate code.

4. Self-documenting APIs

With FastAPI + Pydantic, your API automatically generates Swagger docs that show required fields and types.

MCP Server 2: Advanced Operations

```
# mcp_server_2.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

# Create a FastAPI app instance for MCP Server 2
app_mcp2 = FastAPI(title="MCP Server 2 - Advanced Operations")

# -----
# REQUEST & RESPONSE MODELS (Pydantic)
# -----
# As with MCP Server 1, we use Pydantic models to ensure that all inputs
# and outputs follow a strict schema. This guarantees consistency and
# prevents invalid requests from crashing the server.

class OperationRequest(BaseModel):
    # Input schema: the request must contain two floating-point numbers
    number1: float
    number2: float

class OperationResponse(BaseModel):
    # Output schema: the response always contains
```



```

# - result: numeric result of the operation
# - operation: the operation name (multiplication/division)
# - explanation: a human-readable explanation of what was done
result: float
operation: str
explanation: str

# -----
# TOOL 3: MULTIPLICATION
# -----
# This tool multiplies two numbers together.
# The AI Agent calls it by sending a POST request to /tools/multiply
# with a JSON body matching the OperationRequest schema.

@app_mcp2.post("/tools/multiply", response_model=OperationResponse)
async def multiply_numbers(request: OperationRequest):
    """Multiplication tool - multiplies two numbers"""
    # Pydantic validation ensures number1 and number2 are valid floats.
    result = request.number1 * request.number2

    # The result is returned in a structured JSON format
    return {
        "result": result,
        "operation": "multiplication",
        "explanation": f"Multiplied {request.number1} by {request.number2}"
    }

# -----
# TOOL 4: DIVISION
# -----
# This tool divides one number by another.
# The AI Agent calls it by sending a POST request to /tools/divide
# with a JSON body matching the OperationRequest schema.

@app_mcp2.post("/tools/divide", response_model=OperationResponse)
async def divide_numbers(request: OperationRequest):
    """Division tool - divides first number by second"""

    # Error handling:
    # We explicitly check for division by zero and return a
    # proper HTTP 400 response if detected.
    if request.number2 == 0:
        raise HTTPException(status_code=400, detail="Cannot divide by zero")

    result = request.number1 / request.number2

    return {
        "result": result,
        "operation": "division",
        "explanation": f"Divided {request.number1} by {request.number2}"
    }

```

Why Multiple Servers?

This demonstrates modularity. In production, you might have:

- One server for database operations
- Another for API calls
- A third for file processing
- Each maintained by different teams

Building the AI Agents

AI Agents use language models to understand context and generate human-friendly responses. Our system uses two different agents for different operations.

Agent 1: Kimi K2 for Basic Operations

```
class KimiK2Agent:
    """Agent 1: Kimi K2 for Basic Operations"""

    def __init__(self, base_url: str, token: str, model: str):
        self.client = OpenAI(base_url=base_url, api_key=token)
        self.model = model
        self.name = "Kimi K2 Agent (Basic Ops)"

    def get_response(self, prompt: str) -> str:
        completion = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}],
        )
        return completion.choices[0].message.content
```

Agent 2: OpenAI GPT for Advanced Operations

```
class OpenAIAgent:
    """Agent 2: OpenAI GPT for Advanced Operations"""

    def __init__(self, api_key: str, base_url: str, model: str):
        self.client = OpenAI(api_key=api_key, base_url=base_url)
```

```

self.model = model
self.name = "OpenAI GPT Agent (Advanced Ops)"

def get_response(self, prompt: str) -> str:
    completion = self.client.chat.completions.create(
        model=self.model,
        messages=[{"role": "user", "content": prompt}],
    )
    return completion.choices[0].message.content

```

Agent Design Principles:

- **Single Responsibility:** Each agent handles specific operation types
- **Lazy Loading:** Agents initialize only when needed
- **Abstraction:** Common interface (`get_response`) for different LLM providers

The Orchestration Layer

The Agent API coordinates between users, agents, and MCP servers. This is where the magic happens.

```

# -----
# FUNCTION: get_agent_for_operation
# -----
# This function decides which AI Agent should handle the explanation
# for a given math operation.
#
# Recall: MCP servers only perform raw calculations (e.g., 10 + 5 = 15).
# The AI Agent adds value by generating a *natural language explanation*
# (e.g., "When we add 10 and 5, we get 15 because addition combines values").
#
# In our architecture we have TWO different AI agents:
#   - Kimi K2 Agent → Handles addition & subtraction explanations
#   - OpenAI GPT Agent → Handles multiplication & division explanations
#
# This function acts as a **router** between operations and their
# corresponding explanation agent.

def get_agent_for_operation(operation: str):
    """Route to appropriate AI agent based on operation"""

    # If the operation is addition or subtraction,
    # we want to use the Kimi K2 Agent.
    if operation in ["addition", "subtraction"]:
        # Lazy loading: only load the agent when needed.

```

```

if agent_config["kimi_k2_agent"] is None:
    agent_config["kimi_k2_agent"] = load_kimi_k2_agent()
return agent_config["kimi_k2_agent"]

# Otherwise, the operation must be multiplication or division.
# In that case, we use the OpenAI GPT Agent.
else:
    # Again, we only initialize it once (lazy loading).
    if agent_config["openai_agent"] is None:
        agent_config["openai_agent"] = load_openai_agent()
    return agent_config["openai_agent"]

# -----
# HOW THIS FITS IN THE ARCHITECTURE
# -----
# 1. User enters a calculation in the Streamlit app.
# 2. The Agent API calls the appropriate MCP server tool to get the raw result.
# 3. The raw result (e.g., {"result": 15, "operation": "addition"})
#    is passed here to decide which AI agent should explain it.
# 4. Kimi K2 or OpenAI GPT generates a natural language explanation.
# 5. The Agent API combines tool result + explanation into a final response
#    and sends it back to the frontend.
#
# This design ensures modularity:
# - MCP servers focus only on *calculations*.
# - AI Agents focus only on *explanations*.
# - The Agent API coordinates between them.

```

Tool Calling: The Bridge Between Agent and MCP Servers

This function is the messenger that connects our orchestration layer to the actual math tools. Let's see what happens step by step:

```

# -----
# FUNCTION: call_mcp_tool
# -----
# Purpose:
# This function is responsible for routing a calculation request
# (e.g., "multiply 6 and 7") to the correct MCP server and tool.
#
# Remember:
# - MCP Server 1 → Addition & Subtraction
# - MCP Server 2 → Multiplication & Division
#
# The Agent API itself does NOT perform math. Instead, it delegates
# the actual calculation to the appropriate MCP tool by making an HTTP request.
# -----

```

```

import httpx # HTTP client used to communicate with MCP servers

async def call_mcp_tool(operation: str, number1: float, number2: float):
    """Route to appropriate MCP server"""

    # -----
    # Step 1: Decide which MCP server and tool endpoint to call
    # -----
    if operation in ["addition", "subtraction"]:
        # For addition or subtraction, route to MCP Server 1
        base_url = agent_config["mcp_server_1_url"] # Typically: http://localhost:8001
        # Pick the correct endpoint: /tools/add OR /tools/subtract
        endpoint = "/tools/add" if operation == "addition" else "/tools/subtract"
    else:
        # For multiplication or division, route to MCP Server 2
        base_url = agent_config["mcp_server_2_url"] # Typically: http://localhost:8002
        # Pick the correct endpoint: /tools/multiply OR /tools/divide
        endpoint = "/tools/multiply" if operation == "multiplication" else "/tools/divide"

    # -----
    # Step 2: Make the HTTP POST request to the chosen MCP tool
    # -----
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{base_url}{endpoint}", # Full URL, e.g. http://localhost:8001/tools/add
            json={"number1": number1, "number2": number2}, # Request body
            timeout=agent_config["timeout"] # Configurable timeout
        )

        # Raise an exception if the MCP server returned an error
        response.raise_for_status()

        # Return the parsed JSON response from the MCP tool
        # Example:
        # {
        #   "result": 15,
        #   "operation": "addition",
        #   "explanation": "Added 10 and 5"
        # }
        return response.json()

    # -----
    # HOW THIS FITS IN THE ARCHITECTURE
    # -----
    # 1. The user submits a request via Streamlit (e.g., "10 + 5").
    # 2. The Agent API receives the request and calls this function.
    # 3. This function decides which MCP server to contact.
    # 4. It sends an HTTP POST request with the numbers as JSON.
    # 5. The MCP server performs the calculation and returns the result.
    # 6. The Agent API then passes this raw result to an AI agent
    #    (like Kimi K2 or OpenAI GPT) for natural language explanation.

```

What's actually happening here?



Imagine you're at a restaurant (the Agent API). A customer orders "addition of 10 and 5":

1. **Decision Point:** The waiter (this function) looks at the order and thinks: "Addition? That's handled by the basic math chef (Server 1)"
2. **Routing:** The waiter walks to Server 1's kitchen and goes to the "add" station specifically
3. **Tool Execution:** The waiter hands over the numbers (10 and 5) to that station
4. **Response:** The station calculates 15 and hands back:

```
{  
  "result": 15.0,  
  "operation": "addition",  
  "explanation": "Added 10 and 5"  
}
```

The LLM doesn't see this part at all. The LLM hasn't been involved yet. This is pure Python code making HTTP requests to other servers. The tool does the math, not the AI.

Natural Language Explanations

```
# -----  
# FUNCTION: generate_agent_explanation
```

```

# -----
# Purpose:
# This function takes the raw calculation result from an MCP server
# (e.g., {"result": 15, "operation": "addition"}) and uses an AI agent
# (LLM) to generate a human-friendly explanation of the result.
#
# Why do we need this?
# - MCP servers are great at doing the math, but they only return numbers.
# - End users benefit from context and encouragement (teaching style).
# - This function connects raw math results → natural language insights.
# -----

def generate_agent_explanation(tool_result: dict, operation: str) -> tuple[str,
    """Multi-Agent System: Route to appropriate AI agent for explanation"""

    # Extract the numeric result from the MCP tool output
    result = tool_result["result"]

    # -----
    # Step 1: Decide which AI Agent should handle this explanation
    # -----
    # - For addition/subtraction → Kimi K2 Agent
    # - For multiplication/division → OpenAI GPT Agent
    # This routing is handled by get_agent_for_operation()
    agent = get_agent_for_operation(operation)

    # -----
    # Step 2: Build a natural language prompt
    # -----
    # We instruct the AI agent to act like a friendly math tutor.
    # The goal: explain the calculation result in 2-3 sentences,
    # in a way that's encouraging and easy to understand.
    prompt = f"""You are a friendly math tutor. Explain this calculation result

Operation: {operation}
Result: {result}

Make it easy to understand and add a helpful tip if relevant."""

    # -----
    # Step 3: Get the agent's explanation
    # -----
    # The selected AI agent generates a natural language explanation
    # based on the prompt above.
    explanation = agent.get_response(prompt)

    # Return both the explanation text AND the agent's name
    # (so we know which LLM was used).
    return explanation, agent.name

```

Orchestration Flow:

1. Request arrives with operation type and numbers
2. Route to MCP tool based on operation
3. Execute calculation on appropriate server
4. Route to AI agent for natural language explanation
5. Return enriched response with result and explanation

Complete Request Handler

```
# -----  
# ENDPOINT: /query  
# -----  
# Purpose:  
# This is the MAIN endpoint of the Agent API.  
# It orchestrates the entire multi-agent pipeline:  
# 1. Receives a user query (operation + numbers).  
# 2. Routes the request to the correct MCP server for calculation.  
# 3. Calls the appropriate AI Agent (LLM) to generate an explanation.  
# 4. Combines raw result + explanation into a structured response.  
# 5. Returns this response to the frontend (Streamlit).  
# -----  
  
from datetime import datetime  
import httpx  
from fastapi import HTTPException  
  
@app.post("/query", response_model=AgentResponse)  
async def process_query(request: QueryRequest):  
    """Multi-Agent Processing Pipeline"""  
    try:  
        # -----  
        # Step 1: Call the correct MCP tool  
        # -----  
        # Based on the requested operation (addition, subtraction,  
        # multiplication, division), route to the correct MCP server  
        # and tool endpoint.  
        # Example: "addition" → http://localhost:8001/tools/add  
        tool_result = await call_mcp_tool(  
            request.query,  
            request.number1,  
            request.number2  
        )  
  
        # -----
```



```

# Step 2: Get an AI-generated explanation
# -----
# Once we have the raw tool result (e.g., {"result": 15}),
# we route it to the correct AI Agent (Kimi K2 or OpenAI GPT).
# The agent generates a natural language explanation so the
# user doesn't just see a number, but understands it.
llm_explanation, agent_name = generate_agent_explanation(
    tool_result,
    request.query
)

# -----
# Step 3: Package everything into a response
# -----
# AgentResponse is a Pydantic model that defines the response schema.
# It includes:
# - result: numeric calculation result
# - operation: which operation was performed
# - llm_explanation: friendly natural language explanation
# - agent_used: which AI agent produced the explanation
# - raw_tool_output: original JSON from MCP server
# - timestamp: when the operation was processed
response = AgentResponse(
    result=tool_result["result"],
    operation=tool_result["operation"],
    llm_explanation=llm_explanation,
    agent_used=agent_name,
    raw_tool_output=tool_result,
    timestamp=datetime.now().isoformat()
)

# -----
# Step 4: Cache the result
# -----
# To improve performance, we cache results by creating a unique key.
# Example: "addition_10_5" → stores the response for 10 + 5
# If the same query comes later, we can reuse the cached result.
cache_key = f"{request.query}_{request.number1}_{request.number2}"
calculation_cache[cache_key] = response

# -----
# Step 5: Return the response
# -----
return response

# -----
# Error Handling
# -----
# If MCP server returns an HTTP error (e.g., division by zero),
# raise an HTTPException with the same status code.
except httpx.HTTPStatusError as e:
    raise HTTPException(status_code=e.response.status_code, detail=str(e))

```

```
# Catch-all for unexpected errors in the multi-agent pipeline.
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Multi-agent error: {str(e)}
```

Visualizing the Data Flow

Let's see the complete journey of information through the system:

USER CLICKS "CALCULATE"

↓

Streamlit UI (Frontend)

- Captures: operation="addition", n1=10, n2=5

↓ HTTP POST

Agent API - process_query() [ORCHESTRATOR]

- Receives request
- Starts coordination

↓

↓ ↓

[PHASE 1: COMPUTATION]

↓

call_mcp_tool()

Decision: "addition"

→ Route to Server 1

↓ HTTP POST

MCP Server 1

/tools/add endpoint

Calculates: 10 + 5 = 15

↓ Returns JSON

```
{"result": 15.0,
  "operation": "addition",
  "explanation": "Added 10 and 5"}
```

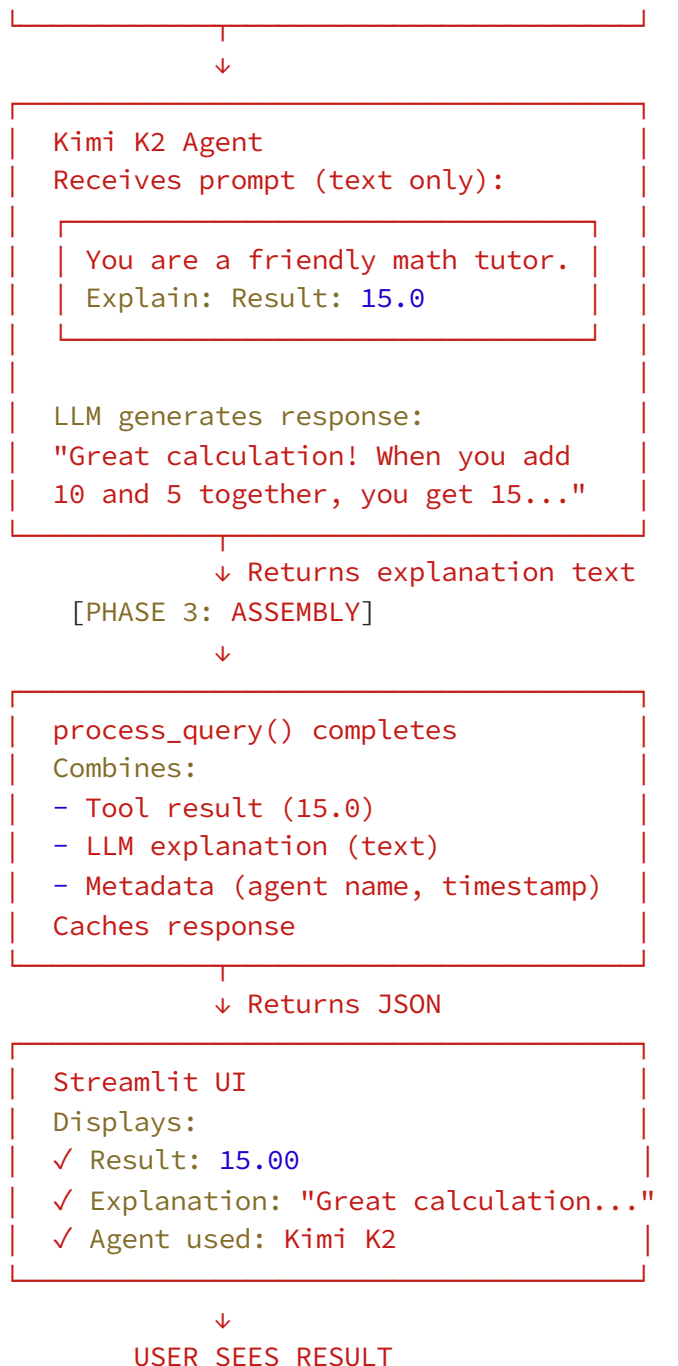
↓

[PHASE 2: AI EXPLANATION]

↓

generate_agent_explanation()

- Extracts result: 15.0
- Selects Kimi K2 Agent
- Builds prompt for LLM



Key Insights About LLM Perception

What the LLM sees: Only text in the prompt

What the LLM doesn't see:

- The HTTP requests
- The MCP servers
- The actual calculation process
- The cache

- Other agents
- The user interface
- Previous calculations (unless we include them in the prompt)

The LLM is stateless: Each request is independent. If you ask “What was the previous result?”, the LLM has no idea unless we explicitly include that information in the new prompt.

Example of LLM’s limited context:

```
# Request 1: 10 + 5 = 15
# LLM sees: "Explain result: 15.0"
# LLM responds: "Great! The sum is 15..."

# Request 2: 20 + 3 = 23
# LLM sees: "Explain result: 23.0"
# The LLM has NO MEMORY of the previous 15!
```

If we wanted the LLM to remember, we’d need to include conversation history:

```
prompt = f"""

You are a friendly math tutor. Previous results:
Addition of 10 and 5 = 15
Now explain this new calculation:
Operation: addition
Result: 23.0

"""
```

Now the LLM can reference the previous calculation because we explicitly told it!

The system uses RESTful HTTP methods appropriately:

GET /health - Check system status (idempotent, no side effects)

POST /query - Submit calculations (creates new result)

PUT /settings - Replace entire configuration (idempotent)

PATCH /settings - Update single setting (partial update)

DELETE /cache - Clear cached results (idempotent)

Configuration Management Examples

```
# -----
# ENDPOINTS: /settings
# -----
# Purpose:
# These endpoints let us update the configuration of the Agent API
# without restarting the server. Configuration includes:
# - API keys for LLM providers (Kimi K2, OpenAI, etc.)
# - Model names
# - MCP server URLs
# - Timeout values
#
# Why is this useful?
# - Credentials or base URLs may change (e.g., rotating API keys).
# - We might want to test different models without redeploying.
# - It makes the system flexible and easy to extend.
# -----

# -----
# PUT - Replace all settings
# -----
# This endpoint updates the entire configuration object at once.
# Use this when you want to refresh multiple values in bulk.
# Example use case: switching from one OpenAI model to another
# while also updating the MCP server URL.
@app.put("/settings")
async def update_all_settings(config: ConfigUpdate):
    """Update entire configuration at once"""

    if config.kimi_k2_token:
        # Update the Kimi K2 API token
        agent_config["kimi_k2_token"] = config.kimi_k2_token

        # Force reload of the agent, so it uses the new token
        agent_config["kimi_k2_agent"] = None

    # ... other settings are updated in the same way
    # (e.g., kimi_k2_model, openai_api_key, timeout, etc.)

    return {"message": "Configuration updated"}

# -----
# PATCH - Update a single setting
# -----
```

```

# This endpoint allows updating ONE configuration field at a time.
# Example: if only the model name changes, you don't have to re-send
# the entire config – just send {"setting_key": "kimi_k2_model",
# "setting_value": "new-model"}.
@app.patch("/settings")
async def update_single_setting(update: PartialConfigUpdate):
    """Update just one configuration value"""

    # Validate that the requested key actually exists in config
    if update.setting_key not in agent_config:
        raise HTTPException(status_code=404, detail="Setting not found")

    # Apply the change
    agent_config[update.setting_key] = update.setting_value

    # If the setting affects agent credentials or model,
    # we must reset the agent so it reloads with fresh config
    if update.setting_key in ["kimi_k2_token", "kimi_k2_model"]:
        agent_config["kimi_k2_agent"] = None

    return {"message": f"Setting '{update.setting_key}' updated"}

```

Key Design Patterns

1. Separation of Concerns

- **MCP Servers:** Handle computation
- **AI Agents:** Provide intelligence and language understanding
- **Orchestration Layer:** Route and coordinate
- **Frontend:** User interaction

2. Lazy Initialization

Agents load only when first needed, reducing startup time and resource usage:

```

if agent_config["kimi_k2_agent"] is None:
    agent_config["kimi_k2_agent"] = load_kimi_k2_agent()

```

3. Error Handling

Graceful degradation with fallback responses:

```
try:
    explanation = agent.get_response(prompt)
except Exception as e:
    print(f"Agent error: {e}, using fallback")
    explanation = f"The sum is {result}."
```

4. Caching

Store results to avoid redundant calculations:

```
cache_key = f"{operation}_{number1}_{number2}"
calculation_cache[cache_key] = response
```

Running the System

Start all components:

```
# Terminal 1: MCP Server 1
uvicorn mcp_server_1:app_mcp1 --port 8001

# Terminal 2: MCP Server 2
uvicorn mcp_server_2:app_mcp2 --port 8002

# Terminal 3: Agent API
uvicorn agent_api:app --port 8000

# Terminal 4: Streamlit UI
streamlit run streamlit_app.py
```

Real-World Applications

This pattern scales to production systems:

Customer Support Agent

- MCP Servers: Ticket system, knowledge base, CRM

- Agents: Route queries, suggest solutions
- Orchestrator: Coordinate multi-step support flows

Data Analysis Agent

- MCP Servers: Database queries, visualization, reporting
- Agents: Interpret results, suggest insights
- Orchestrator: Manage analysis pipeline

DevOps Agent

- MCP Servers: Cloud APIs, monitoring, deployment
- Agents: Diagnose issues, recommend actions
- Orchestrator: Execute multi-service operations

Lessons Learned

Start Simple: Our calculator might seem trivial, but it demonstrates every pattern you'll need for complex systems.

Modularity Wins: By separating MCP servers, agents, and orchestration, each component can evolve independently.

Multiple Agents: Different LLMs have different strengths. Route operations to the best model for each task.

Observability Matters: Log agent decisions, track which tools are called, and monitor performance.

Error Recovery: Always have fallback strategies when agents or tools fail.

Conclusion

AI Agents represent a paradigm shift in software development. By combining the reasoning capabilities of language models with the reliability of traditional APIs through protocols like MCP, we can build systems that are both intelligent and maintainable.

The multi-agent calculator demonstrates these principles at a small scale, but the patterns apply to systems of any complexity. As you build your own agent systems, remember:

- Keep tools focused and single-purpose
- Let agents handle reasoning and coordination
- Use standardized protocols for tool integration
- Design for failure with fallbacks and error handling
- Start simple and add complexity only when needed

The future of software is agentic. Whether you're automating workflows, building intelligent assistants, or creating autonomous systems, understanding these patterns will serve you well.

Next Steps

To extend this project, consider:

- Adding more MCP servers (statistics, graphing, scientific functions)
- Implementing tool discovery (agents learn available tools dynamically)
- Adding memory (agents remember conversation context)
- Building multi-step workflows (chaining multiple operations)
- Implementing agent-to-agent communication
- Adding authentication and rate limiting
- Deploying

The code for this project is available on GitHub. Fork it, experiment, and build something amazing.

Repository Link: [Calculator AI Agent](#)

Connect with me on [LinkedIn](#)

Follow me on [Medium](#)!

+ AI Agent

+ Agentic Ai

+ Agentic Workflow



Follow

Written by Furkhan Suhail

5 followers · 10 following
